1. Processor Fundamentals

1.1. Central Processing Unit (CPU) Architecture

Von Neumann model

- Von Neumann realized data & programs are indistinguishable and can, therefore, use the same memory.
- Von Neumann's architecture uses a single processor.
- It follows a linear sequence of fetch–decode–execute operations for the set of instructions, i.e. the program.
- To do this, the processor uses registers.

Registers

- Registers: smallest unit of storage of microprocessor; allows fast data transfer between other registers
- General Purpose registers
  - Used to temporarily store data values which have been read from memory or some processed result
  - Assembly language instructions can use it
- Special Purpose Registers
  - Some are accessible by assembly language instructions
  - Only holds either data or memory location, not both
  - Particular purpose registers include:
    - Program Counter (PC): holds the address of the next instruction to be fetched
    - Memory Data Register (MDR): holds data value fetched from memory
    - Memory Address Register (MAR): Holds the address of the memory cell of the program which is to be accessed
    - Accumulator (ACC): holds all values that are processed by arithmetic & logical operations.
    - Index Register (IX): Stores a number used to change an address value
    - Current Instruction Register (CIR): Once program instruction is fetched, it is stored in CIR and allows the processor to decode & execute it
    - Status Register: holds results of comparisons to decide later for action, intermediate and erroneous results of arithmetic performed

The Processor (CPU)

- Arithmetic and Logic Unit (ALU): part of the processor that processes instructions which require some form of arithmetic or logical operation
- Control Unit (CU): part of the CPU that fetches instructions from memory, decodes them & synchronizes operations before sending signals to the computer's memory, ALU and I/O devices to direct how to respond to instructions sent to the processor
- Immediate Access Store (IAS): memory unit that the processor can directly access
- System Clock: a timing device connected to a processor that synchronises all components.

Buses

- Set of parallel wires that allow the transfer of data between components in a computer system
  - Data bus: bidirectional bus that carries data instructions between processor, memory, and I/O devices.
  - Address bus: unidirectional bus that carries the address of the main memory location or input/output device about to be used, from processor to memory address register (MAR)
- Control bus
  - Bidirectional
  - used to transmit control signals from the control unit to ensure access/use of data & address buses by components of the system does not lead to conflict

Performance of Computer System Factors

- Clock Speed
  - Number of pulses the clock sends out in a given time interval, which determines the number of cycles (processes) the CPU executes in a given time interval
  - Usually measured in Gigahertz (GHz)
  - If the clock speed is increased, then the execution time for instructions decreases. Hence, more cycles per unit time, which increases performance.
  - However, there is a limit on clock speed since the heat generated by higher clock speeds cannot be removed fast enough, which leads to overheating.
- Bus Width
  - Determines the number of bits that can be simultaneously transferred
  - Refers to the number of lines in a bus
  - Increasing bus width increases the number of bits transferred simultaneously, increasing processing speed and performance.
- Cache Memory
  - Commonly used instructions are stored in the cache memory area of the CPU.
  - If the cache memory size is increased, more commonly executed instructions can be stored, and the need for the CPU to wait for instructions to be loaded reduces. Hence, the CPU executes more cycles per unit of time, thus improving performance.
- Number of Cores
  - Most CPU chips are multi-core — have more than one core (essentially a processor)
  - Each core simultaneously processes different instructions through multithreading, improving computer performance.

Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by at Enderun Colleges on 06/04/26.

Ports

- Hardware which provides a physical interface between a device with CPU and a peripheral device
- Peripheral (I/O) devices cannot be directly connected to the CPU, hence connected through ports.
- Universal Serial Bus (USB): Can connect both input and output devices to the processor through a USB port.
- High Definition Multimedia Interface (HDMI)
  - Can only connect output devices (e.g. LCD) to the processor through a HDMI port
  - HDMI cables transmit high-bandwidth and high-resolution video & audio streams through HDMI ports
- Video Graphics Array (VGA)
  - Can only connect output devices (e.g. second monitor/display) to the processor through a VGA port
  - VGA ports allow only the transmission of video streams but not audio components

Fetch-Execute (F-E) cycle

- Fetch stage
  - PC holds the address of the next instruction to be fetched
  - The address on the PC is copied to MAR
  - PC is incremented
  - Instruction loaded to MDR from the address held in MAR
  - Instruction from MDR loaded to CIR
- Decode stage: The opcode and operand parts of instruction are identified
- Execute stage: Instructions executed by the control unit sending control signals
- Register Transfer Notation (RTN)
  - MAR ← [PC]
  - PC ← [PC] + 1
  - MDR ← [[MAR]]
  - CIR ← [MDR]
  - Decode
  - Execute
  - Return to start
  - *Square brackets: value currently in that register*
  - *Double square brackets:* CPU is getting value stored at the address in the register

Interrupts

- A signal from a program seeking the processor's attention
  - Handles the interrupt by controlling the processor
  - Different ISRs used for different sources of interrupt
- A typical sequence of actions when an interrupt occurs:
  - The processor checks the interrupt register for interrupt at the end of the F-E cycle for the current instruction
  - If the interrupt flag is set in the interrupt register, the interrupt source is detected
  - If the interrupt is low priority, then an interrupt is disabled
  - If interrupting is a high priority:
    - All contents of registers of the running process are saved on the stack
    - PC is loaded with the ISR and is executed
    - Once ISR is completed, the processor pops the registers' contents from the stack, and the interrupted program continues its execution.
    - Interrupts re-enabled and
    - Return to the start of the cycle

1.2. Assembly Language

- Assembly language: low-level programming language with instructions made up of an op code and an operand
- Machine code: code written in binary that uses the processor's basic machine operations
- Relationship between machine and assembly language: every assembly language instruction (source code) translates into exactly one machine code instruction (object code)
- Symbolic addressing
  - Symbols used to represent operation codes
  - Labels can be used for addresses
- Absolute addressing: a fixed address in memory
- Assembler
  - Software that changes assembly language into machine code for the processor to understand
  - The assembler replaces all mnemonics and labels with their respective binary values (that are predefined before by the assembler software)
- One pass assembler
  - Assembler converts mnemonic source code into machine code in one sweep of program
  - Cannot handle code that involves forward referencing

Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by at Enderun Colleges on 06/04/26.

- Two pass assembler: software makes 2 passes through the code
  - On the first pass:
    - Symbol table created to enter symbolic addresses and labels into specific addresses
    - All errors are suppressed
  - On the second pass:
    - Jump instructions access memory addresses via table
    - Whole source code translates into machine code
  - Error reported if they exist
- Grouping the Processor's Instruction Set

Op Code	Operand	Explanation
Addressing		
LDM	#n	Immediate: Load n into ACC
LDD		Direct: load contents at address into the ACC
LDI		Indirect: load contents of address at given address into ACC
LDX		Indexed: load contents of given address + IR into ACC
Data Movement		
STO		Store contents of ACC into address
Arithmetic Operations		
ADD		Add contents of register to ACC
INC		Add 1 to contents of the register
Comparing		
CMP		Compare contents of ACC with that of given address
CMP	#n	Compare contents of ACC with n
Conditional Jumps		
JPE		Jump to address if compare TRUE
JPN		Jump to address if compare FALSE
Unconditional Jumps		
JMP		Jump to given address
I/O Data		
IN	Input any character and store ASCII value in ACC	
OUT	Output character whose ASCII value is stored in ACC	
Ending		
END	Return Control to operating system	

#denotes immediate addressing

B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A

Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by at Enderun Colleges on 06/04/26.

- Modes of Addressing
  - Direct Addressing: loads contents at address into ACC
  - Indirect Addressing: The address to be used is at given address. Load contents of this second address to ACC
  - Indexed addressing: form the address to be used as *+ the contents of the IR (Index Register)*
  - Relative addressing: next instruction to be carried out is an offset number of locations away, relative to address of current instruction held in PC; allows for relocatable code
  - Conditional jump: has a condition that will be checked (like using an IF statements)
  - Unconditional jump: no condition to be followed, simply jump to the next instruction as specified

1.3. Bit Manipulation

- Binary numbers can be multiplied or divided by shifting
- Left shift (LSL #n)
  - Bits are shifted to the left to multiply
  - E.g. to multiply by four, all digits shift two places to left
- Right shift (LSR #n)
  - Bits are shifted to the right to divide
  - E.g. to divide by four, all digits shift two places to right
- Logical shift: zeros replace the vacated bit position
- Arithmetic shift: Used to carry out multiplication and division of signed integers represented by bits in the accumulator by ensuring that the sign-bit (usually the MSB) is the same after the shift.
- Cyclic shift: the bit that is removed from one end by the shift is added to the other end.

Bit Masking

- Each bit can represent an individual flag.
- ∴ by altering the bits, flags could be operated upon.
- Bit manipulation operations:
  - Masking: an operation that defines which bits you want to keep and which bits you want to clear.
  - Masking to 1: The OR operation is used with a 1.
  - Masking to 0: The AND operation is used with a 0.
  - Matching: an operation that allows the accumulator to compare the value it contains to the given value in order to change the state of the status register.
- Practical applications of Bit Masking:
  - Setting an individual bit position:
    - Mask the content of the register with a mask pattern which has 0 in the 'mask out' positions and 1 in the 'retain' positions.
    - Set the result with the match pattern by using the AND command with a direct address.
  - Testing one or more bits:
    - Mask the content of the register with a mask pattern which has 0 in the 'mask out' positions and 1 in the 'retain' positions.
    - Compare the result with the match pattern by using the CMP command or by "Checking the pattern".
  - Checking the pattern
    - Use AND operation to mask bits and obtain resultant.
    - Now subtract matching bit pattern from resultant.

    The final 'non-zero' result confirms the patterns are not the same else vice versa.

Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by at Enderun Colleges on 06/04/26.